

## Tugas Mandiri Modul 6 (DFS)

Nama : Nuevalen Refitra Alswando  
Kelas : IF-04-01  
NIM : 103072430008

### Soal 1 – Pencarian Jalur pada Tree (DFS)

#### Studi Kasus:

Sebuah perusahaan memiliki struktur organisasi berbentuk tree seperti berikut:

- A adalah direktur utama
- B dan C adalah manajer
- D, E adalah bawahan B
- F, G adalah bawahan C

Relasi:

$A \rightarrow B, C$

$B \rightarrow D, E$

$C \rightarrow F, G$

#### Tugas:

1. Buat graph menggunakan **NetworkX** sesuai struktur di atas
2. Implementasikan DFS mulai dari node 'A'
3. Tampilkan urutan edge yang dikunjungi DFS
4. Batasi traversal hanya sampai **depth 2**

#### Output yang diharapkan:

- Urutan DFS tanpa batas
- Urutan DFS dengan `depth_limit=2`

#### Ketentuan:

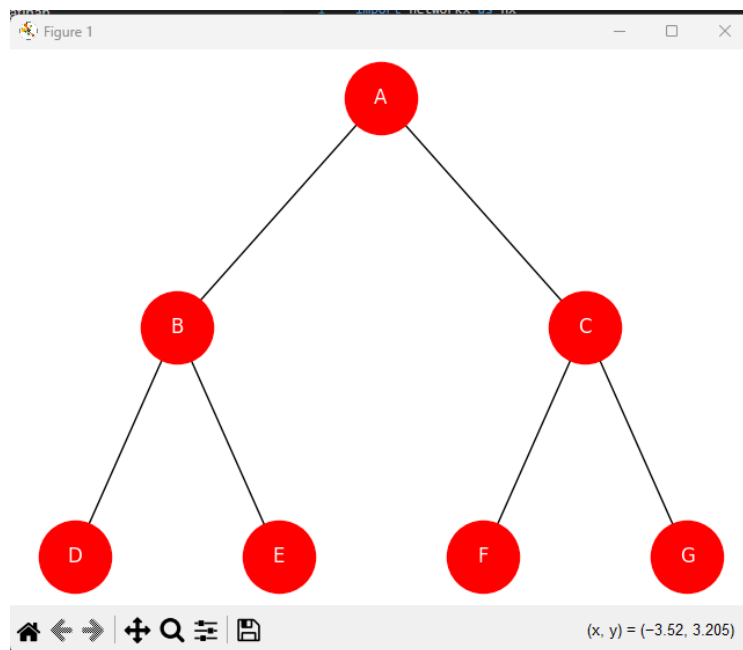
- Gunakan `nx.Graph()`
- Gunakan `nx.dfs_edges()`
- Gunakan perulangan untuk mencetak hasil

## Code

```
modul06 > mandiri > soal1.py > ...
You, 13 hours ago | 1 author (You)
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 G = nx.Graph()
5
6 G.add_edges_from([
7     ('A', 'B'), ('A', 'C'),
8     ('B', 'D'), ('B', 'E'),
9     ('C', 'F'), ('C', 'G')
10 ])
11
12 pos = {
13     'A': (0, 3),
14     'B': (-2, 2), 'C': (2, 2),
15     'D': (-3, 1), 'E': (-1, 1),
16     'F': (1, 1), 'G': (3, 1)
17 }
18
19 nx.draw(
20     G, pos,
21     with_labels=True,
22     node_color='red',
23     node_size=2000,
24     font_color='white'
25 )
26
27 plt.title("Tree DFS")
28 plt.show()
29
30 print("=== DFS Tanpa Depth Limit ===")
31
32 for i, edge in enumerate(nx.dfs_edges(G, source='A'), start=1):
33     print(f"{i}. {edge[0]} -> {edge[1]}")
34
35 print("\n=== DFS Dengan Depth Limit = 2 ===")
36 for i, edge in enumerate(nx.dfs_edges(G, source='A', depth_limit=2), start=1):
37     print(f"{i}. {edge[0]} -> {edge[1]}")
```

```
PS C:\Storage\Documents\Telkom University\Semester 4\Dasar Kecerdasan Artifisial\praktikum\modul06\mandiri> python soal1.py
=== DFS Tanpa Depth Limit ===
1. A -> B
2. B -> D
3. B -> E
4. A -> C
5. C -> F
6. C -> G

=== DFS Dengan Depth Limit = 2 ===
1. A -> B
2. B -> D
3. B -> E
4. A -> C
5. C -> F
6. C -> G
```



Pada codingan ini saya membuat graph berbentuk tree menggunakan library NetworkX dengan node A sebagai root dan node lainnya sebagai cabang. Setelah itu saya menentukan posisi tiap node agar tampilan graph lebih rapi lalu menampilkan graph menggunakan Matplotlib. Selanjutnya saya menerapkan algoritma DFS mulai dari node A untuk menampilkan urutan traversal tanpa batas depth dan traversal dengan batas depth 2 menggunakan `nx.dfs_edges()`.

## Soal 2 – Mencari Jalur ke Tujuan (DFS pada Directed Graph)

### Studi Kasus:

Sebuah sistem navigasi memiliki jalur sebagai berikut:

$A \rightarrow B, C$

$B \rightarrow D, E$

$C \rightarrow E, F$

$E \rightarrow F$

- Node **A** adalah titik awal (Start)
- Node **F** adalah tujuan (Goal)

### Tugas:

1. Buat **directed graph** menggunakan `nx.DiGraph()`
2. Implementasikan DFS mulai dari node 'A'
3. Tampilkan urutan traversal DFS
4. Tentukan apakah node 'F' dapat dicapai menggunakan DFS
5. (Opsional) Hentikan pencarian saat node tujuan ditemukan

### Output yang diharapkan:

- Urutan DFS
- Informasi apakah goal ditemukan atau tidak

### Ketentuan:

- Gunakan `nx.dfs_edges()`
- Gunakan kondisi untuk mengecek apakah node tujuan ditemukan
- Tampilkan hasil dalam bentuk print yang jelas

```

modul06 > mandiri > soal2.py > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 G = nx.DiGraph()
5
6 G.add_edges_from([
7     ('A', 'B'), ('A', 'C'),
8     ('B', 'D'), ('B', 'E'),
9     ('C', 'E'), ('C', 'F'),
10    ('E', 'F')
11 ])
12
13 pos = {
14     'A': (0, 2),
15     'B': (1, 3), 'C': (1, 1),
16     'D': (2, 4), 'E': (2, 2),
17     'F': (3, 2)
18 }
19
20 nx.draw(
21     G, pos,
22     with_labels=True,
23     node_color='red',
24     node_size=2000,
25     font_color='white',
26     arrows=True
27 )
28
29 plt.title("Directed Graph DFS")
30 plt.show()
31
32 print("=== Urutan DFS ===")
33
34 goal = 'F'
35 found = False
36
37 for i, edge in enumerate(nx.dfs_edges(G, source='A'), start=1):
38     print(f"{i}. {edge[0]} -> {edge[1]}")
39
40     if edge[1] == goal:
41         found = True
42         break
43
44 if found:
45     print(f"\nNode {goal} ditemukan")
46 else:
47     print(f"\nNode {goal} tidak ditemukan")

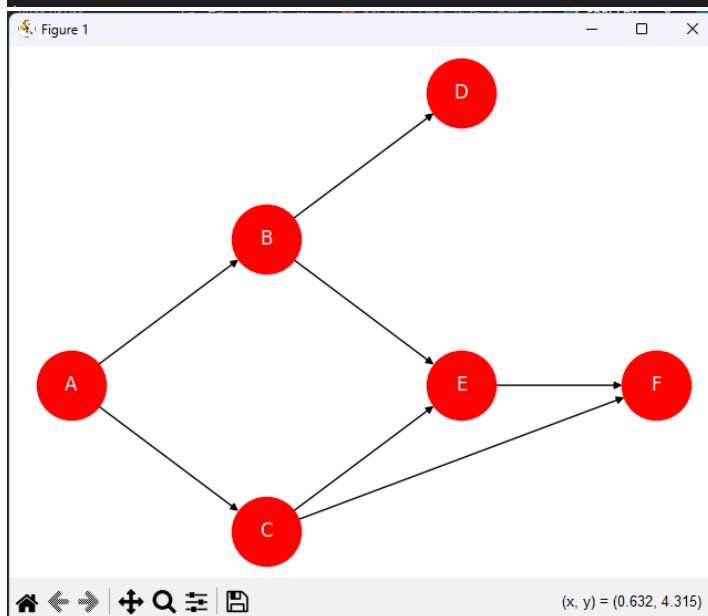
```

```

PS C:\Storage\Documents\Telkom University\Semester 4\Dasar Kecerdasan Artifisial\praktikum\modul06\mandiri> python soal2.py
=== Urutan DFS ===
1. A -> B
2. B -> D
3. B -> E
4. E -> F

Node F ditemukan

```



Pada codingan ini saya membuat directed graph menggunakan library NetworkX dengan node A sebagai titik awal dan F sebagai tujuan. Setelah graph dibuat, saya menampilkan visualisasi graph menggunakan Matplotlib. Selanjutnya saya menerapkan algoritma DFS mulai dari node A menggunakan `nx.dfs_edges()` untuk menampilkan urutan traversal dan mengecek apakah node F dapat ditemukan.